zzgit: An interactive staging, secret-scanning, and commit wizard for zsh

A zsh plugin that replaces `git add . && git commit -am ... && git push` with a deliberate flow, a content-based secret scan of the staged diff, and independent pre-commit and pre-push confirmations.

Ronald G. Thomas

2026-04-22

Figure 1: A vintage mechanical typewriter on a wooden desk, with green terminal-style text emerging on the paper in the carriage. The image juxtaposes a century-old writing tool with the syntax of a modern command-line workflow, echoing the plugin's premise: small ergonomic interventions on a familiar mechanism.

An interactive, content-aware wrapper around `git add`, `git commit`, and `git push` closes several well-known failure modes of the unassisted reflex version of the same three commands.

1 Introduction

I did not really appreciate how much trust I was placing in `git add .` until I watched a colleague accidentally commit an AWS access key embedded in a Jupyter notebook output cell. The notebook was not named `credentials.json`; it was `eda.ipynb`. The key was not flagged by any filename heuristic because its filename was entirely innocuous. The commit landed on a feature branch and was caught later by a scheduled history scan, at which point the key had to be rotated, the commit rewritten, and the remote history force-pushed. None of this would have been necessary if the secret had been caught at the moment of staging rather than after the fact.

The reflex form of the local git workflow, `git add . && git commit -am '...' && git push`, has three well-documented failure modes, each of which contributed to the incident above. It stages every changed or untracked file in the working tree regardless of whether the user intends to publish it. It accepts a commit subject that is typically filename-derived or descriptive of the mechanical action rather than of the user's intent. And it treats the push as an implicit continuation of the commit rather than as a separate decision. Each failure mode is individually minor; in combination they are the proximate cause of most accidentally-committed credentials, of the genre of commit messages that read 'update', and of the specific category of incidents in which a commit is pushed to `main` that the author intended to push to a feature branch.

This post documents a zsh plugin, `zzgit`, that replaces this reflex with a deliberate interactive flow. The plugin stages only files that the user explicitly selects, scans the resulting staged diff for credentials using `gitleaks` (a content-based scanner, not a filename heuristic), assembles a Conventional Commits message through a short wizard, and treats the push as a separately-confirmed action. When the scanner flags a finding, the default behaviour is to refuse the commit and print three remediation paths; the override is guarded by a session-scoped environment variable, requires a typed reason, and is logged. The remainder of the post covers the motivations, the configuration, the design rationale, the limitations, and the set of failure modes that remain.

1.1 Motivations

The design decisions embedded in `zzgit` were motivated by the following specific pain points, each of which the plugin addresses directly:

- **Accidental staging of credentials.** The conventional shortcut `git add .` stages every file in the working tree, including `.env` files, SSH private keys, and downloaded credentials bundles that happen to be present but are not intended to be committed. A filename-based guard (`*.env`, `*credentials*`) catches the most obvious cases but fails on the common class of credentials pasted into normal files: an API key in a configuration

YAML, a database password in a test fixture, a personal access token in a notebook output cell.

- **Commit messages that describe filenames rather than intent.** Under time pressure the author's natural default is to write a commit subject that summarises what files changed, since that information is immediately available from `git status`. This produces a commit history in which the subjects are variants of `fix foo.py bar.py` rather than descriptions of the change's purpose. The history is correspondingly difficult to navigate for bisection, code review, or release-note generation.
- **Pushes that are implicit rather than deliberate.** The sequence `git commit -am '...' && git push` treats publication as a mechanical consequence of the commit. In practice the author should make a second, separate decision at the push step: was the commit assembled correctly, is the branch the intended one, and is the remote ready to receive it. Collapsing the decision increases the rate of inadvertent pushes to the wrong branch and of commits that would have benefited from a moment of review.
- **Protected branches that are only protected server-side.** GitHub and GitLab branch-protection rules are authoritative, but they reject the push at the remote after the author has typed the command. A client-side reminder that `main` is a protected branch closes the specific failure mode of a reflex `git push` from the wrong working copy, even when the remote's protections would ultimately catch the attempt.
- **The absence of a convenient Conventional Commits wizard in the zsh ecosystem.** `commitizen` and `cocogitto` provide the prompting but require Node or Rust toolchains respectively, bring their own configuration conventions, and do not couple to the staging and secret-scanning steps. A single zsh plugin that composes all three concerns is lighter and eliminates the friction of running three separate commands.

1.2 Objectives

The scope of `zzgit` is the following set of verifiable deliverables:

1. Present an interactive list of unstaged and untracked files and stage only the user's selection. Use `fzf` for multi-select when available; fall back to a numeric menu otherwise.
2. Scan the resulting staged diff for credentials using `gitleaks`. When findings are present, refuse to proceed by default and print three remediation options: unstage and fix, allowlist via `.gitleaksignore`, or an audited session-scoped override.
3. Assemble a Conventional Commits message through a four-prompt wizard (type, scope, subject, body) and require a human-written subject as the primary signal of intent.
4. Gate the commit and the push behind independent confirmations. Apply a stronger prompt on pushes to branches listed in `ZZGIT_PROTECTED_BRANCHES`.

The configuration is presented as a reproducible reference rather than as a tutorial; each design decision is annotated with the rationale for its inclusion.

Figure 2: Placeholder ambiance image (coffee, Unsplash free licence). To be replaced with a scene depicting the Conventional Commits wizard: the staged-changes summary, the numbered commit-type menu, and the scope and subject prompts.

2 What is zzgit?

`zzgit` is a zsh plugin of approximately four hundred lines that exposes a single entry point, also named `zzgit`, plus a companion `zzgit-scan-history`. It is structurally a shell function with three internal phases: a selection phase that picks files to stage, a scan phase that inspects the staged diff for credentials, and a message-assembly phase that gathers a Conventional Commits message and requests confirmation before issuing `git commit` and `git push`. The plugin has two soft dependencies, `fzf` and `gitleaks`, each of which degrades gracefully to a reduced-capability fallback when absent.

In functional terms the role of `zzgit` is analogous to that of a pre-commit hook framework, but the integration point is earlier in the workflow and the scope is wider. A pre-commit hook runs after `git commit` has been invoked and operates on a commit that has already been typed; `zzgit` intervenes before the commit is assembled, so that the wizard sees both the user's intent and the scanner's verdict at the same time and can refuse to issue the commit command at all. This matters particularly for the secret-scan path, where a pre-commit hook that refuses a commit still leaves the offending content in the index; `zzgit`'s refusal leaves the index as the user built it and directs them toward remediation without any cleanup.

As a minimal example, invoking `zzgit` inside a clean git working tree with one untracked file containing an `AWS_SECRET_ACCESS_KEY` produces the following path: the plugin lists the untracked file in a selection UI, the user chooses it, `git add` stages it, `gitleaks git --staged` reads the staged diff and flags the key, and the plugin prints the remediation options and exits with status 1 without having issued `git commit`. The credentials never leave the local index and no message was ever typed.

3 Architecture Overview

The runtime system is best understood as three layered components, each with a distinct responsibility:

1. **The wizard function (`zzgit`).** This is the top-level function invoked by the user. It owns the sequential interactive flow, manages the branching on user choices, and is responsible for issuing the `git` subcommands that change repository state. It contains no direct credential-scanning logic; that is delegated to a helper.

2. **The secret-scan helper (`_zzgit_secret_scan`).** This internal function is called once, after staging and before the message-assembly wizard. It selects among three scanning strategies (gitleaks with the newer `git --staged` subcommand, gitleaks with the older `protect --staged` subcommand, or a degraded filename-heuristic fallback when gitleaks is absent), runs the selected strategy, and returns a status code that determines whether `zzgit` proceeds or refuses.

3. **The history-scan companion (`zzgit-scan-history`).** This is a separately-invoked function that runs the scanner over the entire git log rather than the staged diff. It is not on the hot path because history scans on large repositories can take tens of seconds to minutes; it is intended for deliberate use after cloning an unfamiliar repository, after merging a long-lived branch, or as part of a periodic audit.

The invocation path for a typical `zzgit` call therefore traverses: (a) a branch and working-tree status query, (b) an `fzf` or numeric-menu selection, (c) `git add --` on the selection, (d) a `gitleaks` scan of the staged diff, (e) a type-and-subject wizard, (f) a pre-commit confirmation prompt, (g) `git commit`, (h) a pre-push confirmation prompt, and finally (i) `git push`. Each step is interruptible; pressing `Ctrl-C` or answering `n` at any confirmation prompt leaves the index and working tree in a well-defined state.

4 Design Decisions

Six design decisions shape the plugin's behaviour and are worth stating explicitly. Each reflects a trade-off that could reasonably have been decided differently.

The scanner runs on the staged diff, not on the working tree. `gitleaks` has modes for scanning the working tree, the index, and the full history. `zzgit` invokes it on the index (`gitleaks git --staged`) so that the scan sees exactly the content that `git commit` would promote to a commit object. Files that the user chose not to stage are not scanned, which is deliberate: the plugin's scope is the commit that is about to be made, not the developer's broader working environment.

On scan findings, the default is refusal, not a `[y/N]` prompt. A prompt of the form `gitleaks flagged a finding; continue? [y/N]` trains the user to answer `y` on reflex. The `zzgit` design instead returns status 1 on a finding and prints three concrete remediation paths. The override path is gated on an environment variable (`ZZGIT_ALLOW_SECRET_OVERRIDE=1`) that the user must export in the current shell, and further requires a typed reason of at least ten characters. The friction is deliberate and is calibrated against the cost of accidentally publishing a credential.

The subject line is a required human input, not a filename summary. The wizard prompts for a subject before prompting for the body, and if the user leaves the subject blank it falls back to a comma-separated list of the first five staged filenames, matching the behaviour of the original single-function predecessor. Leaving the subject blank is therefore possible but visibly degraded; the prompt structure nudges the user toward writing a human description of the change.

The commit and the push are separately confirmed. After the message is assembled, `zzgit` prints a summary block (branch, staged file list, assembled message) and requires a `y` confirmation before issuing `git commit`. A second, independent confirmation is required before

`git push`. This splits the publication decision cleanly from the commit decision; a user who changes their mind after seeing the diff has an opportunity to cancel without having rewritten any history.

Protected-branch detection is configurable but advisory. The environment variable `ZZGIT_PROTECTED_BRANCHES` defines a space-separated list of branch names that trigger an extra-explicit push prompt. The guard is purely client-side. It does not refuse the push; only the remote can do that. Its value is in catching the specific failure mode of reflexively pushing from the wrong branch, not in enforcing an access-control policy.

The plugin's function definitions are wrapped in `emulate -L zsh`. Each function begins with `emulate -L zsh`, which resets shell options to their zsh defaults for the duration of the function call. This ensures that the plugin behaves identically across users who may have customised `setopt` flags such as `KSH_ARRAYS`, `SH_WORD_SPLIT`, or `NO_UNSET`. The cost is a small increase in function-entry overhead; the benefit is that behaviour is predictable outside the author's own shell.

5 Prerequisites

The configuration assumes the following environment:

- **Shell:** `zsh` 5.0 or later. The plugin uses native zsh array expansion flags (`(f)`, `(j:,:)`) and `typeset -U`; it is not portable to `bash`.
- **Version control:** `git` 2.0 or later. No git features specific to recent releases are used.
- **Operating system:** any POSIX-compatible system on which zsh runs. Development and testing took place on macOS 15; Linux distributions with `zsh`, `git`, and optionally `fzf` and `gitleaks` available in their package repositories are equivalent.
- **Optional runtime dependencies:**
  - `fzf` 0.30 or later for interactive multi-select file staging. Absent: a numeric-menu fallback is used.
  - `gitleaks` 8.0 or later for content-based secret scanning. Versions 8.0 through 8.20 use the `protect --staged` subcommand; versions 8.21 and later use `git --staged`. The plugin detects which form is available and uses it. Absent: a filename-heuristic fallback is used and a prominent installation hint is printed.
- **Plugin manager (optional but recommended):** oh-my-zsh, zinit, antidote, zplug, or sheldon. Manual sourcing from `~/.zsh/plugins/` is supported.
- **Prior knowledge:** familiarity with git workflows, with Conventional Commits, and with zsh environment-variable syntax. No prior knowledge of zsh plugin authorship is required.
- **Time investment:** approximately five minutes for installation and configuration; approximately thirty seconds per invocation thereafter, compared to approximately five

seconds for the `git add . && git commit -am '...' && git push` reflex that `zzgit` replaces.

6 Installation

`zzgit` is distributed as a standard zsh plugin. The recommended installation path depends on the plugin manager in use.

6.0.1 oh-my-zsh

```
git clone https://github.com/rgthomas47/zzgit \
  ${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zzgit
# Add 'zzgit' to the plugins=(...) list in ~/.zshrc, then:
exec zsh
```

6.0.2 zinit

```
# In ~/.zshrc:
zinit load rgthomas47/zzgit
```

6.0.3 antidote

```
# In ~/.zsh_plugins.txt:
rgthomas47/zzgit
```

6.0.4 Manual

```
git clone https://github.com/rgthomas47/zzgit ~/.zsh/plugins/zzgit
# In ~/.zshrc:
source ~/.zsh/plugins/zzgit/zzgit.plugin.zsh
```

Install the optional dependencies (`fzf`, `gitleaks`) through the system package manager; on macOS:

```
brew install fzf gitleaks
```

On Debian-derived Linux distributions `gitleaks` is available through the upstream install script or as a Go-built binary from the project's releases page; the Debian archive version may lag the current release.

7 Configuration

All configuration is exposed as environment variables. There is no configuration file, by design: the configuration surface is intentionally small, and per-repository overrides should be expressed through the repository's own files (`.gitleaks.toml`, `.gitleaksignore`) rather than through a plugin-level file.

7.0.1 Protected branches

```
export ZZGIT_PROTECTED_BRANCHES='main master release develop'
```

Default: `main master`. Any push to a branch whose name matches an entry on this list triggers an extra-explicit confirmation prompt with the branch name surfaced. The guard is advisory; server-side branch protection rules remain the authoritative enforcement layer.

7.0.2 Secret scanner

```
export ZZGIT_SECRET_SCANNER='gitleaks'   # default
export ZZGIT_SECRET_SCANNER='none'       # disable the scan entirely
```

The `none` setting is appropriate for repositories in which upstream CI already runs a blocking scan on every commit, and in which an additional client-side scan would be redundant. In all other cases `gitleaks` is the correct setting.

7.0.3 Override path

```
export ZZGIT_ALLOW_SECRET_OVERRIDE=1    # this shell only
```

When set, a `gitleaks` finding no longer triggers an unconditional refusal; instead the plugin prompts for a reason (at least ten characters) and, on a valid reason, proceeds with the commit and push path. The reason, a timestamp, the user, the branch, and the staged file list are appended to `$GIT_DIR/zzgit-overrides.log`. The variable is deliberately session-scoped: setting it permanently in `~/.zshrc` reduces the guard to a prompt-and-continue, which was the failure mode the design was intended to avoid.

7.0.4 gitleaks configuration

`gitleaks` reads two files from the repository root automatically: `.gitleaks.toml` for rule definitions (additions, modifications, or exclusions) and `.gitleaksignore` for per-finding fingerprints that should be allowlisted. The intended workflow for a confirmed false positive is to paste the fingerprint reported in the scan output into `.gitleaksignore`, commit that file separately with an explanation in the body, then re-run `zzgit`. This mechanism is preferable to the override path because it produces a durable, version-controlled record of the exception.

Figure 3: Placeholder ambiance image (coffee, Unsplash free licence). To be replaced with a scene depicting the secret-scan refusal: a `gitleaks` finding against an AWS access key in a Jupyter notebook's output cell, followed by the three remediation options `zzgit` prints in lieu of a prompt-and-continue.

8 Verification

After installation, the following commands confirm that the plugin is correctly loaded and that the dependencies are in place:

```
# Function presence
whence -w zzgit _zzgit_secret_scan zzgit-scan-history
# Expected output: each name followed by 'function'

# Plugin version
zzgit --version
# Expected output: 'zzgit 0.1.0' (or later)

# Help string
zzgit --help | head -5
# Expected output: a usage summary beginning with 'zzgit - interactive...'

# Dependency probes
command -v fzf       && fzf --version
command -v gitleaks  && gitleaks version
```

A functional smoke test can be run in a throwaway repository:

```
mkdir /tmp/zzgit-smoke && cd /tmp/zzgit-smoke
git init -q -b main
echo 'AWS_SECRET_ACCESS_KEY = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY' \
    > fixture.txt
zzgit    # choose fixture.txt, confirm the scan refuses
cd ~ && rm -rf /tmp/zzgit-smoke
```

The key in the fixture is the published AWS documentation example key, not a live credential; it is recognised by `gitleaks` on the basis of its prefix structure and format.

9 Daily Workflow

After installation the typical invocation pattern is a single command:

```
zzgit
```

The subsequent interaction consists of a short sequence of prompts; the command table below summarises them in the order they appear.

Step	Prompt or action	User input	Effect
1	Working tree status printed	none	informational
2	File selection (fzf or numeric)	TAB / number	populates the stage set
3	`git add --` issued on the selection	automatic	files enter the index
4	`gitleaks` scan of the staged diff	automatic	may refuse
5	Commit type 1-9	number	selects `feat`, `fix`, etc.
6	Scope (optional)	text	becomes `(scope)` in subject
7	Subject (required)	text	headline of the commit
8	Body (optional)	text	detailed description
9	Pre-commit confirmation	`y/N`	gates `git commit`
10	Pre-push confirmation	`y/N`	gates `git push`

The companion command scans the full git history on demand rather than on every invocation:

```
zzgit-scan-history
```

This is appropriate after cloning an unfamiliar repository, after merging a long-lived branch, or on a periodic schedule (weekly or monthly).

10 Things to Watch Out For

Seven gotchas have emerged during use. They are not defects; each represents a deliberate trade-off or a boundary condition that the design choices could not remove.

The protected-branch check is purely advisory. `zzgit` prints a stronger prompt on branches matching `ZZGIT_PROTECTED_BRANCHES` but it does not refuse the push. A user who types `y` on the push prompt will publish to `main` regardless of what the guard says. The guard's purpose is to interrupt reflex, not to enforce policy; for enforcement, configure branch protection on the remote.

Scan coverage is bounded by the installed `gitleaks` rule set. The default rule set covers the major cloud providers, common payment APIs, PEM headers, and several dozen other recognisable credential formats. It does not cover every possible credential format; a bespoke internal API key with no recognisable prefix or entropy signature will pass the scan. Extending `.gitleaks.toml` with project-specific rules is the correct remediation.

Binary blobs can slip through content scanning. A PKCS#12 bundle or an unarmoured SSH key (no `-----BEGIN` marker) does not match any entropy-based or regex-based rule. The degraded filename-heuristic fallback catches these on extension, but the primary scanner does not. Adding path-based rules to `.gitleaks.toml` closes this gap.

Unstaged changes are not scanned. `zzgit` scans the staged diff only. Content in the working tree that the user did not select is not examined. If a secret is present in a file that is not added to the current commit, it will still be present after `zzgit` completes, and will be scanned only on the commit that next stages the file.

The override log is local to each clone. `zzgit-overrides.log` is written to `$GIT_DIR`, which is the local `.git/` directory. It is not committed to the repository and does not follow a clone. If override accountability needs to be centralised, the log should be copied to an auditable location through an external mechanism; this is deliberately not integrated into the plugin.

Filenames containing newlines are unsupported. The zsh array constructs used for file selection split on newlines. A file whose name contains an embedded newline will be treated as multiple entries. This is extremely unusual in practice but possible; on affected repositories the user should stage the offending file manually via `git add --` before invoking `zzgit`.

Detached HEAD resolves to the literal string `HEAD`. The branch-name query `git rev-parse --abbrev-ref HEAD` returns `HEAD` when the repository is in detached-HEAD state. The protected-branch check therefore does not match any of the usual protected branch names, and the push step will fail at the git level regardless of what the plugin does. In practice this is not usually a problem because pushes from a detached HEAD are rare and git itself provides the error.

11 Uninstall and Rollback

The plugin has no persistent state outside its source directory and its environment variables. Removal consists of:

```
# Under oh-my-zsh
rm -rf ${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zzgit
# Remove 'zzgit' from plugins=(...) in ~/.zshrc

# Under zinit
```

```
zinit delete rgthomas47/zzgit

# Manual installation
rm -rf ~/.zsh/plugins/zzgit
# Remove the 'source' line from ~/.zshrc

# Unset environment variables (if exported)
unset ZZGIT_PROTECTED_BRANCHES ZZGIT_SECRET_SCANNER ZZGIT_ALLOW_SECRET_OVERRIDE
```

The per-repository override log (`.git/zzgit-overrides.log`) is left in place on uninstall. It should be archived or deleted manually if no longer needed; it is never committed to the repository's tracked tree.

Figure 4: Placeholder ambiance image (coffee, Unsplash free licence). To be replaced with a scene depicting the post-commit confirmation: the assembled message summary, the commit being made, and the subsequent independent pre-push decision.

12 Lessons Learnt

Working through the design and implementation surfaced a set of lessons grouped into three buckets: conceptual, technical, and gotcha-shaped.

12.0.1 Conceptual

- **Filename heuristics are a poor primary defence against credential leakage.** The filename of the file that contains a credential is essentially uncorrelated with whether the file contains a credential. Content-based scanning is the only defensible primary layer; filename-based checks are useful only as fallbacks when content scanning is unavailable.
- **Confirmation prompts after safety failures are a form of user-experience debt.** A `[y/N]` prompt after a scanner finding trains the user to answer `y`. The friction must be proportional to the risk; for a risk of publishing a credential to a remote, the friction should be high enough that the override is rare and deliberate.
- **The three steps of the local git workflow are not one decision.** Staging, committing, and pushing are logically distinct: what to include, how to describe it, and whether to publish. Collapsing them into a single command (`git add . && git commit -am '...' && git push`) is an ergonomic optimisation that removes two independent decision points that should be preserved.
- **Client-side guards are complementary, not substitutive, to server-side controls.** The plugin's protected-branch guard does not replace GitHub branch-protection rules; it addresses a different failure mode (reflex) than the server-side controls (enforcement). Both layers should be present; neither is sufficient alone.

12.0.2 Technical

- **`gitleaks` scans the index, not just the working tree.** Running `gitleaks git --staged` against the staged diff produces findings for content that is about to enter a commit, regardless of whether the containing file is newly added or previously tracked. This is the right granularity for a pre-commit gate and is strictly stronger than scanning the working tree.
- **Zsh parameter expansion flags subsume most shell scripting conveniences.** `${(f)"$(cmd)"}`, `${(j:,:)array}`, `typeset -U`, and `${array:#pattern}` compose to do what would otherwise require calls out to `awk`, `tr`, `sort`, and `sed`. The resulting code is both faster and less fragile.

- **The `gitleaks` CLI interface changed at version 8.21.** The `protect` subcommand was removed in favour of `git`. A robust invocation probes for the newer subcommand first (`gitleaks git --help`) and falls back to the older form. Hard-coding one form produces a plugin that breaks silently when users upgrade.
- **`emulate -L zsh` inside a plugin function is worthwhile.** It resets the option set to zsh defaults for the duration of the function call, insulating the plugin from user customisations such as `KSH_ARRAYS` or `NO_UNSET`. The overhead is negligible; the robustness gain is substantial for a plugin intended for broad distribution.

12.0.3 Gotcha-shaped

- **`git status --porcelain` column widths differ on renames.** The first two columns plus a space yield the filename column starting at position four for most entries, but rename entries (`R  old -> new`) break the simple `awk '{print substr($0, 4)}'` pattern. Using `git diff --name-only` plus `git ls-files --others --exclude-standard` as separate sources avoids the issue.
- **`head -30` on a piped `git diff --cached` closes the pipe and raises `SIGPIPE` on the producer.** Harmless in interactive zsh but visible in a `set -e` context and in error-handling scripts. `awk 'NR<=30'` reads the entire stream without triggering the pipe close.
- **`git rev-parse --abbrev-ref HEAD` returns `HEAD` in detached-HEAD state.** A naive protected-branch check that compares against `main` and `master` will therefore silently skip the guard in exactly the scenarios where the branch state is unusual. The mitigation is to either add `HEAD` to the protected list or to accept that detached-HEAD pushes will be caught by git itself.
- **Globs in zsh `case` statements match path separators by default.** `*.env` matches `.env`, `foo.env`, and `path/to/.env` in a single pattern because `*` is greedy across `/`. This is the desired behaviour for filename matching but is worth stating explicitly, as it differs from the behaviour of some other shells.

13 Limitations

The design choices embedded in `zzgit` produce the following honest limitations:

- **The plugin does not scan unstaged content.** Files that the user does not select at the staging prompt are never inspected. A credential in an untracked file that is later added in a subsequent invocation will be scanned at that time, but the plugin provides no ambient protection against credentials in the working directory.

- **The scan is only as comprehensive as `gitleaks`'s installed rule set.** Upstream `gitleaks` aggregates rules for known credential formats; bespoke internal formats require project-specific additions in `.gitleaks.toml`. No scanner catches every possible credential shape.
- **The plugin does not verify that findings are live credentials.** `trufflehog` optionally contacts the credential's issuer to confirm that the flagged string is an active credential, which reduces false-positive noise for genuinely suspicious-looking strings that are not in fact credentials. `zzgit` does not use `trufflehog` in v0.1; all findings are treated as equally serious.
- **The override log is not tamper-evident.** An attacker with write access to `.git/` can edit or delete `.git/zzgit-overrides.log`. The log is an accountability record, not an integrity control; for tamper-evidence, the log would need to be mirrored to an append-only store such as a notes tree or an external audit service.
- **The plugin assumes a single remote and a single branch per push.** `git push` is invoked without arguments, relying on the repository's tracking configuration. More elaborate push targets (multiple remotes, explicit refspecs) are not supported; users with such requirements should perform the push manually after `zzgit` completes the commit step.
- **No cross-shell portability.** The implementation uses zsh-native constructs (parameter expansion flags, `typeset -U`) that are not available in `bash`. A port would require substantial rewriting.

14 Opportunities for Improvement

Several extensions are plausible and would be appropriate for subsequent releases:

1. **Completion support.** A `_zzgit` completion file that completes `-h`, `--help`, and `--version` at the command line, plus context-aware completion of commit types and scope candidates. This would make `zzgit` discoverable through zsh's tab-completion infrastructure.
2. **Signed-commit integration.** The plugin currently does not interact with GPG or SSH commit signing. Users with signing configured have `git commit` invoke the signing machinery transparently, which is the correct behaviour; explicit feedback in the pre-commit summary (signing key identifier, expected signature) would make the signing step visible rather than implicit.
3. **Trufflehog verification mode.** An optional mode (`ZZGIT_SCANNER_VERIFY=1`) that, for high-value credentials, runs `trufflehog` in verified mode against the finding. This would distinguish live credentials from credential-shaped strings that are not in fact active, reducing the rate of false-positive overrides.

4. **Repository-level configuration.** Per-repository override files (for example `.zzgit.yaml`) could specify the protected-branch list, the allowed commit types, or a scope-from-path convention, while the plugin-level environment variables govern personal defaults. This would be useful for team repositories in which every developer should see the same protected-branch set.
5. **Pre-commit hook installation.** A helper command `zzgit-install-hook` that installs a `pre-commit` hook into the current repository, wrapping the same secret-scan logic as `zzgit` itself. This would catch commits made through mechanisms other than `zzgit` (for example, through an IDE's git integration), at the cost of adding a layer that some users consider intrusive.
6. **History-scan scheduling.** A wrapper that registers `zzgit-scan-history` with the operating system's scheduler (`launchd` on macOS, `systemd --user` on Linux) to run nightly or weekly, with findings emailed or posted to a designated sink. This would turn the audit from a manual invocation into a standing control.
7. **Structured override-log export.** A command that emits the contents of `.git/zzgit-overrides.log` as JSON, suitable for ingestion into a centralised audit system. The current plain-text format is human-readable but not directly machine-parseable.

15 Wrapping Up

The reflex form of the local git workflow, `git add . && git commit -am '...' && git push`, optimises for typing speed at the cost of three independent failure modes: accidental staging of files the user did not intend to publish, commit subjects that describe filenames rather than intent, and pushes that happen without a second deliberate decision. Each failure mode is individually minor; in combination they account for a substantial fraction of credential-leakage incidents, illegible commit histories, and wrong-branch publications in practice.

The plugin documented here replaces the reflex with an interactive flow that addresses each failure mode directly: an explicit file-selection step rather than `git add .`, a content-based secret scan of the resulting staged diff rather than a filename heuristic, a human-written commit subject rather than an auto-generated file list, and independent pre-commit and pre-push confirmation prompts. The cost is approximately twenty-five additional seconds per commit cycle; the benefit is that the three failure modes are caught at the latest moment at which they can still be caught cheaply, which is before the commit is recorded.

The design intentionally declines several tempting features. It does not refuse pushes to protected branches outright; it prompts more strongly, on the theory that only the remote can authoritatively enforce branch protection. It does not silently allowlist findings that the user overrides; each override is logged with a typed reason. It does not scan the working tree ambiently; its scope is the commit that is about to be made. These choices are subject to revision and are motivated in the Design Decisions section above.

15.0.1 Main takeaways

- **Secret scanning belongs at the staging step, not at push time or in history-scan-only mode.** The latest cheap intervention is before `git commit` is issued.
- **Filename heuristics are a fallback, not a primary defence.** Content-based scanning is the only rule that survives contact with real-world credential placement.
- **Confirmation prompts after safety violations train the wrong behaviour.** Default to refusal; gate the override behind friction calibrated to the risk.
- **The three local-git steps are three decisions.** Preserve them.

16 See Also

- `gitleaks` project repository and documentation: https://github.com/gitleaks/gitleaks
- `fzf` project repository: https://github.com/junegunn/fzf
- Conventional Commits specification: https://www.conventionalcommits.org/
- `commitizen` (Node-based alternative to the Conventional Commits wizard portion): https://commitizen-tools.github.io/commitizen/
- `cocogitto` (Rust-based alternative, also enforces commit-message validity): https://docs.cocogitto.io/
- Related qblog post on zsh terminal configuration (post 01): `/posts/01-configtermzsh/`

17 Reproducibility

The configuration was developed and verified on the following software stack:

Component	Version	Notes
Operating system	macOS 15 (Sequoia)	Linux equivalents in package repositories
Shell	zsh 5.9	minimum 5.0
Version control	git 2.43	minimum 2.0
Plugin version	zzgit 0.1.0	tagged release
`fzf`	0.46.1	optional; 0.30+ supported
`gitleaks`	8.21.2	optional; 8.0+ supported, pre-8.21 uses `protect` subcommand
Plugin manager	oh-my-zsh 2026-04	zinit / antidote / manual source also tested

Date of last verification: 2026-04-22.

18 Let's Connect

Corrections, questions about specific design decisions, and bug reports are welcome through the plugin's repository issue tracker. Contributions should follow the repository's contribution guidance; pull requests that change the default refusal behaviour on scan findings will be reviewed with particular care, as that decision is load-bearing for the plugin's security posture.